AI DATA ANALYST AGENT USING STREAMLIT, SQLITE AND OLLAMA

PROJECT DETAILS

Attribute	Description
Project Title	AI Data Analyst Agent Using Streamlit, SQLite and Ollama
Project Domain	Artificial Intelligence & Data Analytics
Project Type	Natural Language to SQL Query System
Frontend Technology	Streamlit
Backend Technology	Python
Database	SQLite
AI Engine	Ollama
Language Model	Llama 3
Development Environment	VS Code
Operating System	Windows/Linux

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

The AI Data Analyst Agent is an intelligent application that allows users to interact with databases using natural language. Instead of writing complex SQL queries, users can simply ask questions in English and receive results instantly.
The system uses Artificial Intelligence and Large Language Models (LLMs) to convert user queries into SQL statements. These queries are executed on a SQLite database, and the results are displayed in tabular and graphical formats.

CHAPTER 2: PROBLEM STATEMENT

Traditional database systems require technical expertise in SQL and database structures. Non-technical users often struggle to retrieve information from databases efficiently.

Problem Statement

Develop an AI-powered data analytics system that converts natural language queries into SQL commands and presents the results through visualizations and tables.

CHAPTER 3: OBJECTIVES

Objective No.	Objective
1	Enable database querying using natural language
2	Generate SQL queries automatically
3	Execute SQL queries securely
4	Provide graphical visualization of results
5	Store query history
6	Improve accessibility for non-technical users
7	Implement local AI processing
8	Reduce dependency on manual SQL writing

CHAPTER 4: LITERATURE SURVEY

Existing System	Limitation
Traditional SQL Systems	Requires SQL expertise
Tableau	Requires understanding of data structures
Power BI	Limited natural language flexibility
AI Chatbots	Lack direct database integration

Research Gap

Dependence on cloud-based AI
Lack of secure SQL validation
Limited local deployment options
High technical dependency

CHAPTER 5: SYSTEM ANALYSIS

Existing System

User → SQL Query → Database → Output

Drawbacks

- Requires SQL knowledge
- Time-consuming
- Error-prone
- Technical dependency

Proposed System

User → Natural Language Query → AI Agent → SQL Query → Database → Visualization

Benefits

- Easy to use
- Faster analytics
- Automated visualization
- Reduced learning curve

CHAPTER 6: TECHNOLOGY STACK

Technology	Version	Use in Project
Python	3.x	Core programming language used for backend development and AI integration
Streamlit	Latest	Creates interactive web-based user interface
SQLite	Latest	Stores and manages application data locally
Ollama	Latest	Executes Large Language Models locally
Llama 3	Latest	Converts natural language into SQL queries
Pandas	Latest	Data manipulation, cleaning, and processing
Matplotlib	Latest	Generates charts and visualizations
Requests	Latest	Handles communication with Ollama APIs
SQL	Standard	Query language for database operations
VS Code	Latest	Development and debugging environment

Technology Justification

Technology	Reason for Selection
Python	Simple syntax and extensive AI libraries
Streamlit	Rapid web application development
SQLite	Lightweight and serverless database
Ollama	Supports local AI execution
Llama 3	High-quality natural language understanding
Pandas	Efficient data analysis
Matplotlib	Easy chart generation

CHAPTER 7: SYSTEM ARCHITECTURE

Layer	Function
User Layer	Accepts user queries
Presentation Layer	Streamlit Interface
AI Layer	Query interpretation and SQL generation
Validation Layer	SQL safety checking
Database Layer	Data storage and retrieval
Visualization Layer	Generates charts and reports

Workflow

User enters query.
AI interprets the query.
SQL statement is generated.
Safety validation occurs.
Query executes on SQLite database.
Results are displayed.
Charts are generated automatically.

CHAPTER 8: MODULE DESCRIPTION

Module Name	Purpose
app.py	Main Streamlit application
agent.py	Controls workflow and AI communication
sql_generator.py	Generates SQL from user questions
schema_reader.py	Reads database schema
safety_checker.py	Prevents unsafe SQL execution
query_executor.py	Executes database queries
visualizer.py	Creates graphs and charts
history_manager.py	Maintains query history

CHAPTER 9: DATABASE DESIGN

Database Information

Item	Description
Database Name	database.db
Database Type	SQLite
Storage Type	Embedded

Sample Table Structure

Field Name	Data Type	Description
ID	INTEGER	Primary Key
Name	TEXT	Employee Name
Department	TEXT	Department Name
Salary	INTEGER	Employee Salary

CHAPTER 10: IMPLEMENTATION METHODOLOGY

Step	Description
1	User submits query
2	Schema information retrieved
3	SQL generated using AI
4	Query validated
5	SQL executed
6	Results fetched
7	Charts generated
8	History saved

CHAPTER 11: USER INTERFACE DESCRIPTION

Interface Component	Function
Query Input Box	Accepts user questions
Execute Button	Runs the query
SQL Display	Shows generated SQL
Result Table	Displays data
Visualization Area	Shows charts
Sidebar	Displays history and settings

CHAPTER 12: TESTING AND VALIDATION

Test Case	Expected Output	Status
Database Connection	Connected Successfully	Pass
SQL Generation	Valid SQL Query	Pass
Query Execution	Correct Results	Pass
Visualization	Chart Generated	Pass
History Storage	Record Saved	Pass

CHAPTER 13: ADVANTAGES AND LIMITATIONS

Advantages

Advantages
Natural language interaction
No SQL expertise required
Local AI processing
Automatic chart generation
Secure query execution
User-friendly interface

Limitations

Limitations
Depends on AI model accuracy
SQLite unsuitable for huge datasets
Requires Ollama installation
Schema-dependent performance

CHAPTER 14: FUTURE SCOPE

Future Enhancement
PostgreSQL Support
MySQL Integration
SQL Server Support
Cloud Deployment
User Authentication
Voice-Based Querying
PDF Report Generation
Real-Time Analytics
Dashboard Customization
Multi-User Access

CHAPTER 15: CONCLUSION

The AI Data Analyst Agent successfully bridges the gap between non-technical users and database systems. By integrating Python, Streamlit, SQLite, Ollama, and Llama 3, the project enables intelligent data retrieval and visualization through natural language queries. The system enhances accessibility, productivity, and decision-making while providing a secure and efficient data analytics platform.
